

Assignment #1: Linear Data Structures

Conestoga College - ACSIT

Algorithms & Data Structures (PROG2340)

Due: Friday Oct 9, 2026 (by Midnight)

Total marks: 40 Worth: 10%

Introduction

The goal of this assignment is to apply your Python & DSA skills and knowledge to a simple console-based application.

Grading

This assignment will be graded from 3 different perspectives. First, because you are being asked to write runnable code that outputs specific results the most significant focus of the grade is on whether or not it runs correctly and produces the correct output. Thus each of the required 11 features (as determined by the 11 menu options) are each graded out of 2 where:

Grade	Description
2	The solution: runs without errors; produces a result; and the result meets the specified criteria exactly.
1	The solution runs and produces output but there is at least some aspect of the output criteria that has not been met. Or, a poor or minimal solution was submitted. For instance, there might be some code provided but it either fails to run or runs with errors.
0	No solution submitted or no code attempted in the solution.

And each of the 2 conceptual algorithm analysis questions the following scheme will be used:

Score	Level	Description
4	Exemplary	Both complexities and descriptions were correct.
3	Proficient	Both complexities were correct but 1 description was lacking.
2	Developing	Both complexities were correct but both descriptions were lacking.
1	Beginning	Only 1 complexity was correct and both descriptions lacking.
0	Not done	No answer submitted.

And finally the code you author will be evaluated from the following code quality perspectives:

Code characteristic	Weight	Description
Naming, Formatting & readability	2	The variable, function, class, method, module, etc. names are meaningful and add to readability and clarity. And the code spaced well and separated into cohesive chunks of code.
Modularity & structure	2	The solution makes appropriate use of functions, classes & methods, modules, etc.
Used correct techniques	6	Techniques used were appropriate for the task at hand and were what was explicitly asked for in the question. This also means that you cannot utilize techniques that were not covered in class.

General comments

You can make use of as many Python files as you want/need but the file that kicks off the console-based app **must** be called `main.py`.

Also, a couple of points about my grading process:

- I will be running a script to run and evaluate your code
 - So you need to be very precise in your prompts, responses, and the order on options and inputs collected.
 - For example, as described below, when adding a new book you need to prompt the user for inputs in the exact order specified which in this case is:
 - ISBN
 - Title
 - Author
- Then that will be followed by the script opening up your responses in the text file(s) for me to read those, if there are any.

Problem Description: an eBook Store

You are managing a bookstore's catalog. Implement a console-based, menu-driven app that allows a user to interact with the bookstore program. The menu must include the following options and in this exact order and numbered exactly as you see here:

1. Add Book to Catalog
2. Display Catalog
3. Search Books by Author
4. Search Books by Title
5. Search Book by ISBN
6. Remove Book by ISBN
7. Add Book to Checkout Cart
8. Remove Book from Cart
9. Peek Cart
10. Display Cart Size
11. Exit Program

The program should continue running until the user selects the *Exit* option.

Detailed requirements (22 marks)

1. Add Book to Catalog

- Prompts user (in this order) for:
 - ISBN
 - Title
 - Author
- Adds the book to the catalog
- Rejects duplicate ISBNs
- Sample Output:
 - Book 'The Alchemist' by Paulo Coelho added successfully.
- Or:
 - A book with ISBN 123 is already in the collection.

2. Display Catalog

- Displays the entire catalog.
- Sample Output:
 - "The Alchemist" by Paulo Coelho (ISBN: 111)
 - ...
 - "Atomic Habits" by James Clear (ISBN: 222)
- Or:
 - No books to display, please add some!

3. Search Books by Author

- Prompts for an author's name and displays all books by that author
- Sample Output:
 - "The Alchemist" by Paulo Coelho (ISBN: 111)
 - "Another Book" by Paulo Coelho (ISBN: 444)
- Or:
 - No books by Bart Simpson

4. Search Books by Title

- Prompts for a title and displays all matching books
- Sample Output:
 - Books titled 'Deep Work'..
 - "Deep Work" (ISBN: 333)
- Or:
 - No books with that title

5. Search Book by ISBN

- Prompts for ISBN and displays book details if found
- Sample Output:
 - Found "Atomic Habits" by James Clear
- Or:
 - No book found with that ISBN

6. Remove Book by ISBN

- Prompts for ISBN and removes that book if it exists
- Sample Output:
 - Removed "Atomic Habits" by James Clear

- Or:
 - No book found with that ISBN
- 7. Add Book to Checkout Cart
 - Prompts for a title and adds it to the checkout stack
 - Sample Output:
 - Book 'The Hobbit' added to cart.
- 8. Remove Book from Cart
 - Removes the last book from the cart (LIFO)
 - Sample Output:
 - Removed 'Dune' from cart.
- 9. Peek Cart
 - Shows the last book in the cart without removing it
 - Sample Output:
 - Top of cart: '1984'
- 10. Display Cart Size
 - Displays the total number of books in the cart
 - Sample Output:
 - Cart contains 2 book(s).
- 11. Exit
 - Ends the program
 - Sample Output:
 - Exiting Bookstore System, goodbye!

Implementation Details (10 marks)

Use the following considerations in your implementation. Also, it is recommended to use helper functions for input validation (e.g. a valid menu option) and catalog display.

Doubly Linked List (Book Catalog)

Implement a *"Doubly Linked List"* to keep track of books with the following considerations in mind.

- Each Node should store:
 - isbn (string, unique)
 - title (string)
 - author (string)
- Your list should support the following operations:
 - add_book(isbn, title, author)
 - Add a book to the catalog, grouped by author
 - If the author already exists, insert after their last book
 - Otherwise, add to the end
 - ISBNs must be unique
- get_books_by_author(author)
 - Return all books by a given author as a linked list (empty list if none found)
- get_books_by_title(title)
 - Return all books with a given title as a linked list (empty list if none found)
- get_books_by_isbn(isbn)
 - Return a single book matching the ISBN (or None if not found)

- `remove_book_by_isbn(isbn)`
 - Remove and return the book with the given ISBN (or None if not found)
- `__str__()`
 - Converts the catalog to string (i.e. from head --> tail separated by <-->)

The following code snippet will be helpful:

```
catalog = BookCatalog()
catalog.add_book(BookNode("111", "The Alchemist", "Paulo Coelho"))
catalog.add_book(BookNode("222", "Atomic Habits", "James Clear"))
catalog.add_book(BookNode("333", "Deep Work", "Cal Newport"))
catalog.add_book(BookNode("444", "Another Book", "Paulo Coelho"))
```

And when you print out the catalog the output (grouped by author) should look like the following:

```
"The Alchemist" by Paulo Coelho (ISBN: 111)
"Another Book" by Paulo Coelho (ISBN: 444)
"Atomic Habits" by James Clear (ISBN: 222)
"Deep Work" by Cal Newport (ISBN: 333)
```

Stack (Customer Checkout)

At checkout, books chosen by a customer are placed in a **Stack** (i.e. so that the last added is scanned first). For this **Stack** data structure you can use only simple string, i.e. the title of the book, but the idea is that you can only add books that you actually have in the catalog. That is, before you add a title to the **Stack** first make sure that it is indeed in the catalog!

Implement the stack for checkout with the following operations:

- `add_to_cart(title)`
 - Add a book to the stack (cart)
- `remove_from_cart()`
 - Remove and return the last book added (or None if empty)
- `peek_cart()`
 - View the last book in the cart without removing it (or None if empty)
- `cart_size()`
 - Return the number of books in the cart
- `is_cart_empty()`
 - Check if the cart is empty

The following code snippet will be helpful:

```
cart = CheckoutCart()
cart.add_to_cart("The Hobbit")
cart.add_to_cart("1984")
cart.add_to_cart("Dune")
```

```
print(cart.remove_from_cart())  
print(cart.peak_cart())  
print(cart.cart_size())
```

Which would output:

```
Dune  
1984  
2
```

Algorithm Analysis (8 marks)

In a simple text file called `analysis.txt` (which needs to be in the same directory as `main.py`) include an algorithm analysis for the following:

- `add_book` method
- `remove_book_by_isbn` method

Include:

- The time & space complexities
- And a brief explanation for each of those

Submission

The Python file(s) must contain heading with student information. Once you have your files (named correctly where applicable) then zip them up into a single zip file named: `-assignment1.zip`. For example, suppose we have a user named Sally Jones whose Conestoga email address is `Sjones3412@conestogac.on.ca` (and note: this also means that `Sjones3412` matches her Conestoga domain username), then create a single zip file named as follows: `"Sjones3412-assignment1.zip"`. Again, it is very important that it is both:

- Named in exactly this format,
- `main.py` & `analysis.txt` are in the same directory, and
- A zip file (i.e. not a tar, rar, etc..) that when unzipped will give me all the properly named files for the assignment
 - i.e. those files should not then be in a sub-directory.

Then submit that correctly named single zip file to the eConestoga dropbox for the assignment. You can submit multiple solutions but only the latest (i.e. most recent) one will be looked at and evaluated.